

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 7

Team No.: 7

Project Title: Linux Process Resource Monitoring System

Team Members

Roll Number	Student Name	Signature
2520030177	P. Harini	
2520030178	Ashrita	
2520030077	P. Bindhu Madhavi	

1. Abstract

The **Linux System Information and Resource Monitoring Tool** is a Linux-based system programming project developed using the C programming language. The system is designed to monitor important operating system resources and running processes and display the collected information through a simple command-line interface. It provides system-level information including CPU details, overall CPU usage, memory usage, disk usage, and system uptime. The tool also scans the Linux /proc filesystem to obtain information about currently running processes, including their process ID, process name, process state, and memory usage. Users can configure CPU and memory thresholds according to their requirements. When the monitored resource usage exceeds the configured threshold, the system generates an appropriate warning. The project also provides process control using Linux signals, allowing the user to stop, continue, or terminate a selected process using its PID. During continuous monitoring, the system tracks process activity and identifies newly created and terminated processes. Important monitoring activities and user operations are stored in a log file for future reference. The project follows a client-server approach using TCP socket communication, where the client provides the user interface and the server performs the monitoring operations. POSIX threads are used to support continuous monitoring. The project demonstrates practical Linux and Operating Systems concepts including /proc filesystem handling, process monitoring, signals, file I/O, threads, sockets, and system resource management.

2. Problem Statement

A Linux system may run a large number of processes at the same time, making it difficult to observe system activity and resource usage manually. Processes may consume significant CPU or memory resources, remain active for a long time, or change their state during system operation. The user needs a simple mechanism to view system information and monitor running processes without repeatedly checking multiple Linux commands. There is also a need to identify when configured CPU or memory limits are exceeded and provide a clear warning to the user. In addition, when a particular process needs attention, the user should be able to control that process using its process ID. The system should also provide information about processes that are newly created or terminated during monitoring and maintain a record of important activities. To address these requirements, the project develops a C-based Linux monitoring tool that directly obtains system and process information from the Linux /proc filesystem and other Linux system interfaces. The tool displays CPU, memory, disk, uptime, and process information through a command-line client. It allows CPU and memory thresholds to be configured, generates warnings when limits are exceeded, provides signal-based process control, tracks process activity, and stores monitoring events in log files. A TCP client-server mechanism is used to communicate between the user interface and the monitoring server, while continuous monitoring is supported using POSIX threads.

3. Objectives

1. To monitor running Linux processes and their states.
2. To collect and display CPU and memory usage information.
3. To obtain system and process information from the Linux /proc filesystem.
4. To display CPU details, disk usage, and system uptime.
5. To configure CPU and memory usage thresholds.
6. To generate warnings when resource usage exceeds the configured thresholds.
7. To provide process control using Linux signals.
8. To stop, continue, or terminate a selected process using its PID.
9. To track newly created and terminated processes.
10. To maintain logs of important monitoring activities.
11. To provide continuous monitoring using POSIX threads.
12. To provide client-server communication using TCP sockets.
13. To provide a simple command-line interface for monitoring and managing the system.

4. Proposed Methodology

The project is developed using the **C programming language on Ubuntu/Linux**. The system follows a client-server approach in which the client provides a command-line interface and communicates with the monitoring server using TCP socket communication. The server collects system information such as CPU details, CPU usage, memory usage,

disk usage, and system uptime using Linux system interfaces. It also scans the Linux /proc filesystem to obtain information about running processes, including their PID, name, state, and memory usage. The user can configure CPU and memory thresholds, and the system checks the current resource usage against these limits and generates warnings when the configured limits are exceeded. The system provides process control using Linux signals, allowing a selected process to be stopped, continued, or terminated using its PID. During continuous monitoring, a POSIX thread performs monitoring operations and checks changes in running processes to identify newly created and terminated processes. Important system activities, threshold changes, resource checks, and process-control operations are recorded in the monitor.log file. The client provides menu options to view system information, process information, configure thresholds, check resources, control processes, start or stop continuous monitoring, and view logs.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
System Calls	Used to interact with the Linux operating system and obtain system and process information.
TCP Socket Communication	Used for communication between the client and monitoring server. The client sends commands and receives monitoring results from the server.
/proc Filesystem	Used to read live system and process information such as CPU usage, memory usage, process ID, process name, and process stat
Signals (kill(), signal handling)	Used to control selected processes using SIGSTOP, SIGCONT, and SIGTERM.
statvfs()	Used to obtain filesystem information and calculate disk usage.
File I/O	Used to create and maintain the monitoring log file and store important system and process activities.
POSIX Threads	Used to perform continuous monitoring of system resources and process activity while the server continues handling client requests.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030178	Ashritha	Process log management, /proc data extraction, continuous monitoring, testing and integration

Roll Number	Student Name	Individual Responsibility
2520030177	P. Harini	Process control and signalling using SIGSTOP, SIGCONT, and SIGTERM, TCP client-server communication, threshold monitoring and testing
2520030077	P. Bindhu Madhavi	System information, CPU and memory monitoring, process information and command-line interface

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Operating system used for developing, testing, and running the process monitoring tool.
C Programming Language	Language used to implement process monitoring, signalling, and logging logic.
GCC Compiler	To compile and build the C source code.
Linux Terminal	To run the tool and display live process resource information.
GitHub	To store, version, and submit the project source code and documentation.

8. Expected Outcome

The project produces a functional Linux System Information and Resource Monitoring Tool with a command-line interface for monitoring and managing system resources and running processes. The system displays CPU information, overall CPU usage, memory usage, disk usage, system uptime, and process information including PID, process name, process state, and memory usage. It allows the user to configure CPU and memory thresholds and generates warnings when the configured limits are exceeded. The system also provides process control using Linux signals, allowing selected processes to be stopped, continued, or terminated. During continuous monitoring, the system tracks newly created and terminated processes and records important monitoring activities in a persistent log file. The project uses the Linux /proc filesystem, system calls, file I/O, POSIX threads, and TCP socket communication between the client and monitoring server. The completed implementation demonstrates practical application of Linux Operating Systems and Systems Programming concepts through a working C-based monitoring system.

9. GitHub Repository

GitHub Repository Link:

<https://github.com/pakalapatiharini/-Linux-Process-Resource-Monitoring-System>

Repository Created: ☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: _____

Signature: _____

Remarks: _____

Faculty Use

Project Approved: ☐ Yes ☐ No

Date: _____

Faculty Signature: _____